

Tested Features

Antivirus

- Pathfinding
- Killing player
- Not collecting data/source code/firewall/key

Player

- Movement
 - No moving into walls
- Collecting data
- Collecting source code
- Damaging by firewall
 - Losing (data < 0) from firewall damage
- Exiting to win
 - Cannot exit when data left over

Collectables

- Source code spawning
 - Source code expiry
- Decryption key collection
 - Door unlocking
- Firewall spread
 - Blocked by walls, other entities
 - Stopped when frozen
- Freeze token collection
 - Causes firewall spread freezing

UI Menu and Rendering

- Title menu shows at game start.
 - Animated background frames
- Options menu shows on “Options” button click
 - Title menu shows on “Back” button click
 - Volume can be set, parametrized testing with values from 0, 25, 75, 100
- Starting game on “Start Mission” button click
 - Tutorial appears and dismissable with “Got it” button
- In game, pause menu activated with ESC key press or pause icon button click
 - Pause menu dismissed on “Resume” button click

- Title menu shows on “Exit” button click
- In game, player movement and interaction
 - Player sprite moves with W A S D pressed and released
 - Server room door opens when key collected
 - Screen flash on damage taken
- In game’s HUD stats
 - Elapsed time and score correctly updated per board tick
 - “Exit Unlocked” notification shown once when all data collected
 - Progress bar shown when a freeze item collected
- Win and lose screen correctly showing info
- Animated entity (player, antivirus)
 - Correct animated frames used while heading left/right
 - Idle frame used while not moving
 - Fallback to idle if animated frames missing
- Asset and audio management
 - Caching assets on a map
 - Playing audio on set volume
 - Asset fallback on missing files, null stream, exception

Component Interactions

Application launches to TitleMenu (VirusBreachTest.startsOnTitleMenu)

Full JavaFX application starts via TestFX -> active menu is the TitleMenu

TitleMenu “Start Mission “ -> GameMenu (VirusBreachTest.canStartGame)

Clicking the Start Mission button via TestFX robot transitions the active menu from TitleMenu to GameMenu.

Board Generation -> valid entity and tile placement (BoardGeneratorTest)

After generateGoard, 6 Data Packets, 10 Firewalls, 1 Antivirus, and 1 Player are present. No random walls in room interiors, buffer zones, or doorways. The server room door is initially locked.

Player collects any Collectable -> data score updates (CollectableTest.playerCollection)

Player moves onto any collectable entity -> collectable is removed from the board -> players data score is updated.

**Antivirus cannot collect any Collectable
(CollectableTest.noAntivirusInteraction)**

Antivirus moves onto any collectable entity -> no score change

**Antivirus + A* pathfinding -> pursues Player through the Board
(AntivirusTest.navigateLeft/Right/Up/Down)**

Antivirus uses A* pathfinding on the Boards tile grid to locate and move toward the Player each tick cycle.

Antivirus navigates around walls (AntivirusTest.navigateAround, navigateFar, pathfindAroundLarge)

When a wall blocks the path of the antivirus, it will use A* to find an alternate route without stepping on the wall tiles.

Player + Firewall -> data decrease -> game lost (FirewallTest.canKillPlayer)

Player moves onto a Firewall -> Firewalls -200 penalty is applied -> if data drops below zero, onLose() is triggered

**Board freeze timer -> Firewall spread suppressed
(FirewallTest.cannotSpreadFrozen)**

Boards freeze timer is set -> Firewalls tick() checks the freeze state -> spreading is suppressed for the duration

**Player + FreezeToken -> Board freeze timer starts
(FreezeTokenTest.collectFreezeToken)**

Player moves onto the FreezeToken -> token is removed -> boards freeze timer becomes non-zero to pause all firewall spread.

**SourceCode expires -> Player cannot collect it
(SourceCodeTest.sourceCodeExpiredUncollectable)**

SourceCode ticks down and is removed from the board -> Player then moves onto that tile -> no data is awarded

**Player + DecryptionKey -> LockedDoor becomes OpenDoor
(DecryptionKeyTest.unlockDoor)**

Player moves onto the DecryptionKey tile -> key is removed from the board -> the LockedDoor tile at the door position is replaced with an OpenDoor tile, physically opening the server room.

Antivirus cannot OpenDoor (DecryptionKeyTest.noAntivirusInteraction)

Antivirus moves onto the DecryptionKey -> key stays on board -> door remains LockedDoor.

Player cannot exit without all Data**(ExitTest.cannotEscapeWithoutCollectingData)**

Player steps on the Exit tile while Data entries still exist on the board -> win is not triggered.

Player exits to win after collecting all data**(ExitTest.canEscapeAfterCollectingData)**

Player collects remaining data and steps on Exit -> win is triggered.

Player exits without collecting SourceCode**(ExitTest.canEscapeWithoutCollectingSourceCode)**

SourceCode (bonus collectible) is not required to exit – only required Data Items block the exit

Antivirus cannot trigger Exit (ExitTest.antivirusCannotExit)

Antivirus steps on Exit tile -> win is not triggered

Die and restart (GameMenuTest)

Player loses the game -> GameOverMenu is shown -> clicking restart transitions back to a new GameMenu with a fresh board.

Pause and quit (GameMenuTest)

ESC key press activates PauseMenu -> clicking Exit transitions back to TitleMenu.

Ensuring Test Quality

While implementing the game, we separated the game (board) logic from the rendering of the game, which allowed us to have all game logic run asynchronously on a separate thread. This practice has made testing easier for most of the tests, since we don't need to initialize JavaFX to simulate the game, except for test cases that target UI components.

In each test for game logic, `TestHelper` is used to create an enclosed board that is simple for testing and is independent from other tests. Each functional test targets a behaviour of the game and assert observable outcomes as a result of the behaviour.

For UI tests, we used the TestFX library that gives us a `robot` to click and read UI components. Since the testing thread and the JavaFX application thread run asynchronously, the function `waitForFxEvents()` is used to wait for events after an action (e.g. clicking) before asserting. TestFX also provides an assert function `verifyThat` to match visible nodes to the expected UI state. Explicit setup and cleanup are added to prevent shared JavaFX state from affecting other tests.

Code Coverage

100% Full line and branch coverage on:

- Root: HackingGame, GameOptions
- Math: Direction, Position
- Board: Room, ServerRoom, StorageRoom, BoardObserver
- Tile: Exit, Entrance, LockedDoor, Floor, OpenDoor, Wall, TileType
- Entity: Antivirus, Player, Firewall, Entity, SourceCode, DecryptionKey, Collectable, FreezeToken, Data
- UI: AnimatedNode, AudioManager, ResourceManager, TutorialOverlay, HUD
- UI Menu: GameMenu, WinMenu, GameOverMenu, TitleMenu, OptionsMenu, PauseMenu, Menu

Almost full coverage on:

- Root: BoardGenerator (90% branch coverage). The remaining missed branches are due to short-circuit evaluation of compound `&&` conditions inside `placeWall()`. JaCoCo counts each sub-expression of a compound condition as a separate branch. For 100% coverage, tests for trivial or irrelevant input are needed, which does not add meaningful test value.
- Board: Board class is 95% covered due to a condition only targeted for testing. This class shuts down the runtime of the UI library JavaFX when an exception occurs. Some of our tests, however, intentionally cause exceptions. This led to subsequent tests not being able to start since a JavaFX runtime is a singleton per JVM. An if statement is created to detect whether the game is under testing, and if so, would not shut down the JavaFX runtime.

Findings

Some of our production code was changed during the testing phase to fix revealed bugs, improve code quality, and assist with testing.

Production changes made to fix bugs:

- Firewall spread freezing state being able to leak out into new boards
- Handling missing asset fallback on animated entity

Production changes made to improve code quality:

- Removed dead code in decryption key unlocking

Production changes made to improve testability:

- Added package-private getters and setters to initially internally private states that were needed to be asserted
- Made some methods package-private instead of fully private
- As mentioned in test coverage, the JavaFX runtime shutdown is conditioned to not run during testing
- Changes to board entity management